

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	17
REFERÊNCIAS.....	18

EMSE

O QUE VEM POR AÍ?

Nesta aula, você aprenderá sobre o conceito de Imagens Docker e sua estrutura, além de criar e gerenciar imagens customizadas. Também iremos entender o que é registry e qual sua finalidade, incluindo como fazer upload de imagens customizadas para um registry público bem famoso.

Prepare-se para uma jornada que irá mudar sua perspectiva sobre desenvolvimento e implantação de aplicações.

EMANIP

HANDS ON

Nessa aula você aprenderá como realizar download, listagem, remoção e criação de imagens Docker. Você também executará containers com imagens customizadas e criar uma conta no Docker Hub.

EXEMPLO

SAIBA MAIS

Clarificando o que é imagens e registries

Imagens no mundo dos containers são binárias e há um conjunto de instruções destinadas à execução de softwares sem interface gráfica. Essa execução é feita dentro do container de forma isolada por meio da imagem, podendo possuir N configurações de acordo com a situação problema a ser resolvida.

O mais interessante é que podemos criar nossas próprias imagens com configurações customizadas, seja para compartilhar com outras pessoas ou para usarmos quando quisermos.

Registry nada mais é do que um centralizador em que podemos realizar upload de imagens ou baixar imagens já existentes para os nossos projetos. Assim, ele é uma ferramenta essencial para gerenciar, compartilhar e guardar imagens de forma pública ou privada.

Diferença de containers e imagens

Container é um ambiente isolado em que podemos definir por qual porta TCP podemos realizar vinculação para gerar comunicação com nossa máquina local para a execução de aplicações.

Já imagens docker são pacotes imutáveis que contém N camadas em sua estrutura, sendo cada uma delas uma configuração específica, contendo assim tudo o que é necessário para executar uma aplicação, incluindo bibliotecas, variáveis, arquivos de configurações e códigos.

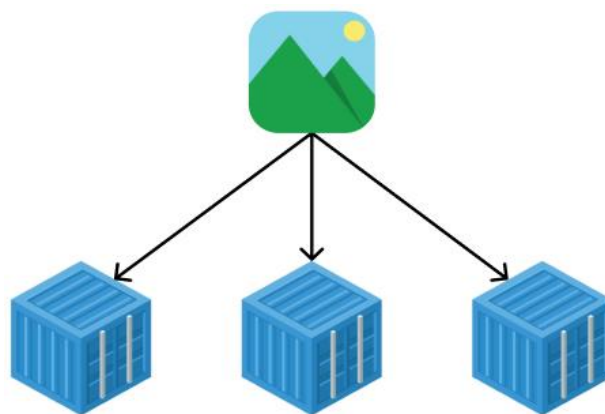


Figura 1 - Demonstração de como uma imagem pode ser utilizada para vários containers
Fonte: Elaborado pelo autor (2024)

Fica claro, portanto, que quando um container docker é criado a partir de uma imagem, estamos na verdade criando uma instância isolada dessa imagem. Em outras palavras, estamos convertendo uma imagem docker para um container.

Assim, um container nada mais é do que uma instância em execução de uma imagem, um ambiente isolado em que a aplicação (ou container) roda conforme as configurações definidas pela imagem.

Estrutura de imagens docker

Imagens docker, como dito anteriormente, são imutáveis e formadas por N camadas, cada uma representando uma configuração ou modificação no sistema de arquivos.

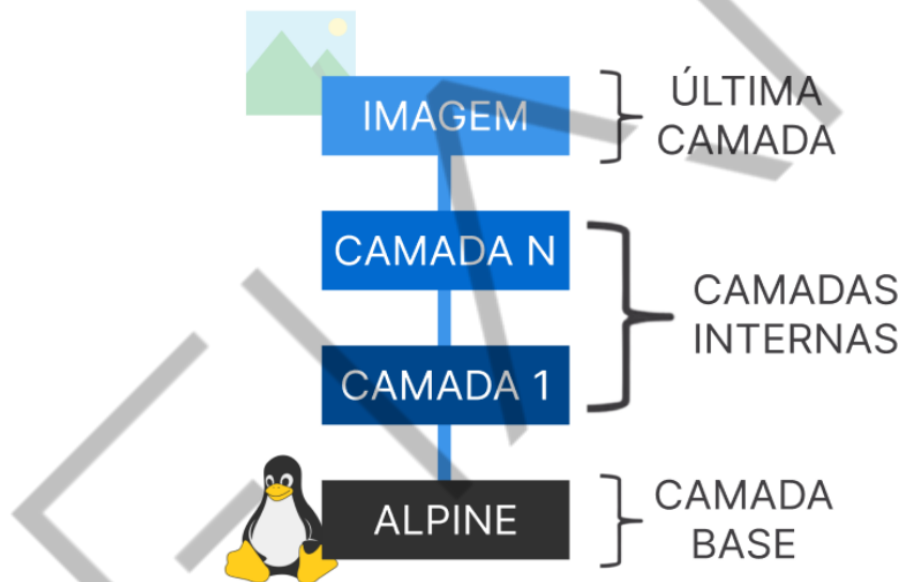


Figura 2 - Demonstração de como é a estrutura de uma imagem
Fonte: Elaborado pelo autor (2024)

A camada base geralmente é do tipo Alpine Linux, mas também temos outras como, por exemplo, debian ou ubuntu. A Alpine Linux é conhecida por ser muito leve e minimalista, tendo o necessário para a configuração de uma imagem; sendo assim, você também pode criar sua imagem docker utilizando a Alpine Linux.

Para realizar o pull (ou download) de uma imagem Alpine, podemos utilizar o seguinte comando:

```
$ docker pull alpine
```

Código-fonte 1 – Exemplo de comando que baixa uma imagem do registry
 Fonte: Elaborado pelo autor (2024)

Após realizar o comando docker pull, vemos que a imagem Alpine só tem uma camada, sendo ela a d25f557d7f31:

```
nb9935@localhost:~$ docker pull alpine
Using default tag: latest
latest: Pulling from library/alpine
d25f557d7f31: Pull complete
Digest: sha256:b2bf705cebbbc5a03a0b58e49ec152261ea7da707f40f7d5db1c4f75ed72d43c1
Status: Downloaded newer image for alpine:latest
docker.io/library/alpine:latest
```

Figura 3 - Demonstração da saída do download de uma imagem do registry
 Fonte: Elaborado pelo autor (2024)

Para confirmar o número inicial de camadas, podemos executar o seguinte comando, que tem como função exibir informações referentes a volumes, containers, networks e imagens:

```
$ docker inspect <image_id ou image_name>
```

Código-fonte 2 – Exemplo de comando que exibe as informações de uma imagem docker
 Fonte: Elaborado pelo autor (2024)

Podemos ver na chave **Layers** (camadas) que temos somente uma camada no array e essas camadas se encontram dentro do **RootFS** (ou diretório principal de arquivos):

```
{
  "Volumes": null,
  "WorkingDir": "",
  "Entrypoint": null,
  "OnBuild": null,
  "Labels": null
},
  "Architecture": "amd64",
  "Os": "linux",
  "Size": 7794716,
  "GraphDriver": {
    "Data": {
      "MergedDir": "/var/lib/docker/overlay2/232563b7bb106a78e093d829afd9eb5a0a28b6dffbac3092d30a0b8d0b6df794/merged",
      "UpperDir": "/var/lib/docker/overlay2/232563b7bb106a78e093d829afd9eb5a0a28b6dffbac3092d30a0b8d0b6df794/diff",
      "WorkDir": "/var/lib/docker/overlay2/232563b7bb106a78e093d829afd9eb5a0a28b6dffbac3092d30a0b8d0b6df794/work"
    },
    "Name": "overlay2"
  },
  "RootFS": {
    "Type": "layers",
    "Layers": [
      "sha256:02f2bcb26af5ea6d185dcf509dc795746d907ae10c53918b6944ac85447a0c72"
    ]
  },
  "Metadata": {
    "LastTagTime": "0001-01-01T00:00:00Z"
  }
}
```

Figura 4 - Demonstração da saída do comando docker inspect
 Fonte: Elaborado pelo Autor (2024)

Além disso, podemos ver o histórico da imagem para entender o que está sendo executado em cada camada e quais alterações (ou adições, lembrando que uma imagem é imutável) foram feitas e transformadas em novas camadas. Para isso, podemos utilizar o seguinte comando:

```
$ docker history <image_id ou image_name>
```

Código-fonte 3 – Exemplo de comando que exibe informações de cada camada da imagem
Fonte: Elaborado pelo autor (2024)

Após executar o comando anterior, vamos ter o seguinte resultado para a imagem Alpine:

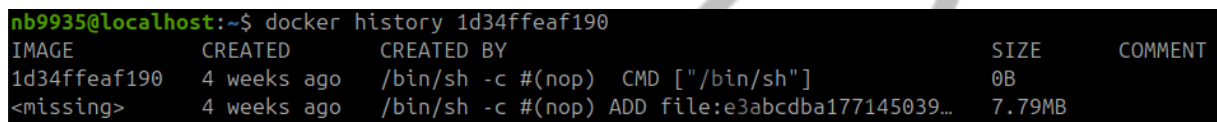


IMAGE	CREATED	CREATED BY	SIZE	COMMENT
1d34ffeaf190	4 weeks ago	/bin/sh -c #(nop) CMD ["/bin/sh"]	0B	
<missing>	4 weeks ago	/bin/sh -c #(nop) ADD file:e3abcdba177145039...	7.79MB	

Figura 5 - Demonstração detalhada do que se passa em cada camada
Fonte: Elaborado pelo autor (2024)

Repare que temos as colunas IMAGE, CREATED, CREATED BY, SIZE e COMMENT. A coluna IMAGE representa as camadas, se tem hash. Logo, a linha desse hash é uma camada da nossa imagem; caso contrário, se a linha na coluna IMAGE está como **<missing>**, isso indica que a camada base da imagem Alpine foi removida do sistema local após a construção da imagem.

A próxima coluna, CREATED, nos diz o tempo decorrido desde a criação dessa imagem. Já a coluna CREATED BY exibe os comandos de execução utilizados, passo a passo, para a criação da imagem. Já a coluna SIZE nos diz o tamanho de cada camada. E, por fim, a coluna COMMENT serve para deixar algum comentário sobre a camada.

Agora vamos criar uma imagem customizada a partir da imagem Alpine para vermos a criação de uma nova camada. Para isso execute o seguinte comando:

```
$ docker run --name <container_name> -it alpine
```

Código-fonte 4 – Exemplo de comando que executa uma imagem no modo iterativo
Fonte: Elaborado pelo autor (2024)

Após executar o comando, vamos criar um arquivo fictício de configuração para fins de demonstração de como alterar uma imagem base:


```
nb9935@localhost:~$ docker run --name meu-alpine -it alpine
/ # touch minha_configuracao.conf
/ # echo "Essa é minha configuração" > minha_configuracao.conf
/ # cat minha_configuracao.conf
Essa é minha configuração
/ #
```

Figura 6 - Demonstração da alteração feita dentro do container
Fonte: Elaborado pelo autor (2024)

Após isso, podemos **digitar exit** para **sair do modo iterativo**. Em seguida, podemos criar nossa imagem. Para isso, precisamos listar os containers, pegar o ID do container e realizar o commit do container. A seguir temos os comandos utilizados.

```
$ docker ps -a
```

Código-fonte 5 – Exemplo de comando que exibe todos os containers inativos
Fonte: Elaborado pelo autor (2024)

```
$ docker commit <container_id> meu-user-dockerhub/meu-alpine
```

Código-fonte 6 – Exemplo de comando que salva o estado atual do container como imagem
Fonte: Elaborado pelo autor (2024)

```
$ docker images
```

Código-fonte 7 – Exemplo de comando que exibe todas as imagens do docker host
Fonte: Elaborado pelo autor (2024)

Quando realizamos o **commit do container**, estamos **criando uma imagem a partir da versão atual do container**; é assim, que criamos novas camadas, pois a imagem dentro do container se torna “flexível”, na qual conseguimos adicionar novas camadas. O container nada mais é que a instância da imagem. Nesse caso, a instância da imagem, sendo ela o container, foi alterada com sucesso. Sendo assim, teremos o seguinte resultado:

```
nb9935@localhost:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS          NAMES
19867e6015ed   alpine    "/bin/sh"               9 minutes ago   Exited (130) 2 minutes ago           meu-alpine
514d8a3a0058   nginx     "/docker-entrypoint..." 9 minutes ago   Exited (0) 9 minutes ago           trusting_jepsen
nb9935@localhost:~$ docker commit 19867e6015ed meu-user-dockerhub/meu-alpine
sha256:6fc5d341351d9f1106d7e97a41291bd4a692b250cf901b3096b952a374917b13
nb9935@localhost:~$ docker images
REPOSITORY          TAG         IMAGE ID      CREATED        SIZE
meu-user-dockerhub/meu-alpine   latest     6fc5d341351d  3 seconds ago  7.79MB
```

Figura 7 - Demonstração da conversão do estado atual do container para imagem
Fonte: Elaborado pelo autor (2024)

Após a realização do commit, é importante gerar as tags para gerenciamento de versões; no mundo Docker gerenciamos as imagens dessa maneira. Com o

comando a seguir, podemos gerar nossa tag a partir da imagem (ou estado atual do container) que acabamos de comitar.

```
$ docker tag meu-user-dockerhub/meu-alpine meu-user-dockerhub/meu-alpine:1.0
```

Código-fonte 8 – Exemplo de comando que cria tag para a imagem (1)
Fonte: Elaborado pelo autor (2024)

Na sequência, também devemos gerar a tag latest como boas práticas. Para isso é necessário executar o seguinte comando:

```
$ docker tag meu-user-dockerhub/meu-alpine meu-user-dockerhub/meu-alpine:latest
```

Código-fonte 9 – Exemplo de comando que cria tag para a imagem (2)
Fonte: Elaborado pelo autor (2024)

O resultado ficará da seguinte maneira:

```
nb9935@localhost:~$ docker tag meu-user-dockerhub/meu-alpine meu-user-dockerhub/meu-alpine:1.0
nb9935@localhost:~$ docker tag meu-user-dockerhub/meu-alpine meu-user-dockerhub/meu-alpine:latest
nb9935@localhost:~$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
meu-user-dockerhub/meu-alpine	1.0	6fc5d341351d	8 minutes ago	7.79MB
meu-user-dockerhub/meu-alpine	latest	6fc5d341351d	8 minutes ago	7.79MB

Figura 8 - Demonstração de boas práticas de como as imagens devem ser taguadas
Fonte: Elaborado pelo autor (2024)

Agora vamos verificar quantas camadas temos em nossa imagem Alpine customizada. Para isso, execute novamente o comando:

```
$ docker inspect <image_id ou image_name>
```

Código-fonte 10 – Exemplo de comando que exibe todas as informações de uma imagem
Fonte: Elaborado pelo autor (2024)

Teremos o seguinte resultado:

```
{
  "OnBuild": null,
  "Labels": {}
},
{
  "Architecture": "amd64",
  "Os": "linux",
  "Size": 7794870,
  "GraphDriver": {
    "Data": {
      "LowerDir": "/var/lib/docker/overlay2/232563b7bb196a78e093d829afd9eb5a0a28b6dffbac3092d30a0b8d0b6df794/diff",
      "MergedDir": "/var/lib/docker/overlay2/97ceda95060dff5fd8ede1a2382f57555c11cd1e99e5c1bafa79708ba3ee5dc7/merged",
      "UpperDir": "/var/lib/docker/overlay2/97ceda95060dff5fd8ede1a2382f57555c11cd1e99e5c1bafa79708ba3ee5dc7/diff",
      "WorkDir": "/var/lib/docker/overlay2/97ceda95060dff5fd8ede1a2382f57555c11cd1e99e5c1bafa79708ba3ee5dc7/work"
    },
    "Name": "overlay2"
  },
  "RootFS": {
    "Type": "layers",
    "Layers": [
      "sha256:02f2bcb26af5ea6d185dcf509dc795746d907ae10c53918b6944ac85447a0c72",
      "sha256:e3a4ef533182dd200c574e4262089834913474289f2c4412cb1756dba71dbc1b"
    ]
  },
  "Metadata": {
    "LastTagTime": "2024-06-20T15:50:57.069529378-03:00"
  }
}
```

Figura 9 - Demonstração da saída do comando docker inspect
Fonte: Elaborado pelo autor (2024)

Podemos verificar que agora temos duas camadas, pois realizamos a inserção de uma nova configuração na imagem base Alpine.

E se executarmos o comando de histórico da imagem, vamos ver que tivemos também uma atualização:

```
$ docker history meu-user-dockerhub/meu-alpine
```

Código-fonte 11 – Exemplo de comando que exibe informações de cada camada da imagem
Fonte: Elaborado pelo autor (2024)

Veja que o resultado do comando de histórico da imagem nos retornou uma nova linha com um hash. Sendo assim, podemos confirmar que todas as linhas que tiverem hash na saída do comando de histórico de uma imagem são as camadas de uma imagem.

```
nb9935@localhost:~$ docker history meu-user-dockerhub/meu-alpine
IMAGE          CREATED          CREATED BY          SIZE      COMMENT
6fc5d341351d   21 minutes ago  /bin/sh             154B
1d34ffeaf190   4 weeks ago    /bin/sh -c #(nop)  CMD ["/bin/sh"]     0B
<missing>      4 weeks ago    /bin/sh -c #(nop)  ADD file:e3abcdba177145039... 7.79MB
```

Figura 10 - Demonstração detalhada do que se passa em cada camada
Fonte: Elaborado pelo autor (2024)

Registries

Registry é uma aplicação server-side em que podemos armazenar e gerenciar imagens. Quando uma imagem é publicada no Registry, um repositório é criado automaticamente para armazenamento e organização do histórico de versão. Isso tem relação com o tópico anterior em que executamos os comandos a seguir:

```
$ docker tag meu-user-dockerhub/meu-alpine meu-user-dockerhub/meu-alpine:1.0
```

Código-fonte 12 – Exemplo de comando que cria tag para a imagem (1)
Fonte: Elaborado pelo autor (2024)

```
$ docker tag meu-user-dockerhub/meu-alpine meu-user-dockerhub/meu-alpine:latest
```

Código-fonte 13 – Exemplo de comando que cria tag para a imagem (2)
Fonte: Elaborado pelo autor (2024)

Criamos as tags para o gerenciamento de versão para o Registry. Para publicá-las nele, devemos executar os seguintes comandos (é necessário criar uma conta [aqui](#) para a criação do seu usuário):

```
$ docker login
```

Código-fonte 14 – Exemplo de comando que faz login no docker hub
Fonte: Elaborado pelo autor (2024)

```
$ docker push meu-user-dockerhub/meu-alpine:1.0
```

Código-fonte 15 – Exemplo de comando que publica uma nova imagem no registry (1)
Fonte: Elaborado pelo autor (2024)

```
$ docker push meu-user-dockerhub/meu-alpine:latest
```

Código-fonte 16 – Exemplo de comando que publica uma nova imagem no registry (2)
Fonte: Elaborado pelo autor (2024)

Quando executamos o comando `docker push`, estamos solicitando um método de criação no Registry para o nosso usuário criar um novo repositório e armazenar nossa imagem estática. Após esses passos, dependendo das configurações de gerenciamento do repositório, essa imagem fica disponível para download.

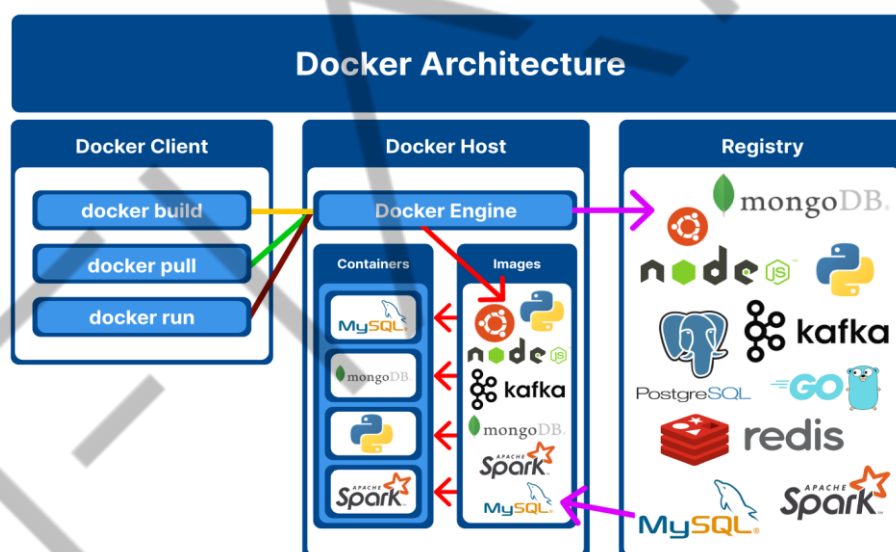


Figura 11 - Demonstração da arquitetura do docker
Fonte: Elaborado pelo autor (2024)

Como podemos ver na figura acima, quando executamos um comando do Docker em nosso terminal é realizada uma solicitação para o docker engine. Essa solicitação pode ser de criação, remoção, download etc.

Quando precisamos baixar uma nova imagem para o nosso docker host (docker local), o docker engine faz uma requisição via REST-API para realizar download da imagem (seta lilás).

Caso a imagem já exista no docker host, o docker engine tem a inteligência de identificar que a imagem específica já existe em nosso sistema, logo a requisição não é feita para o Registry e o container é criado em sequência. Ou se executarmos o comando `docker pull`, é possível identificar que a imagem existe e uma mensagem parecida com a figura a seguir que será exibida:

```
nb9935@localhost:~$ docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
Digest: sha256:56b388b0d79c738f4cf51bbaf184a14fab19337f4819ceb2cae7d94100262de8
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest
```

Figura 12 - Demonstração da saída de uma imagem já existente no docker host
Fonte: Elaborado pelo autor (2024)

Registries do mercado atual

Docker Hub

A Docker inventou o formato padrão OCI para a entrega de containers juntamente com a adoção por todos os principais sistemas operacionais, fazendo sentido que o Docker Hub seja também o registro padrão para gerenciamento de imagens. Se você está no desenvolvimento, é provável que já tenha usado o Docker Hub, especialmente se já seguiu um de nossos materiais.

Embora todos os serviços de registro nesta lista ajudem você a gerenciar o formato da Docker, o Docker Hub como registro ainda merece um lugar aqui. Não apenas porque oferece um enorme registro público que você pode usar para implantar aplicativos de fornecedores grandes e pequenos, mas também para entregar os seus próprios.

Amazon ECR

O Amazon's Elastic Container Registry (ECR) é útil se você já está usando a Amazon Web Services (AWS) para hospedar aplicativos devido a sua integração e opções de gerenciamento de equipe.

O ECR da Amazon está integrado a todos os seus serviços de hospedagem de contêineres. Então, você pode gerenciar e enviar facilmente suas imagens para:

- Elastic Container Services (ECS).

- Elastic Kubernetes Services (EKS).
- AWS Lambdas.

Assim como o Docker Hub, seu registro público e mercado também valem a pena, permitindo a implantação de muitos produtos, software gratuito e projetos de código aberto. Você nunca estará sem ambientes existentes para construir.

Harbor

O Harbor é um registro de código aberto que você pode instalar quase em qualquer lugar, mas particularmente adequado para Kubernetes.

Enquanto certos serviços amarram seus registros aos seus próprios serviços, a liberdade do Harbor o torna uma opção versátil. Ele é compatível com a maioria dos serviços em nuvem e plataformas de Integração Contínua e Entrega Contínua (CI/CD), além de também ser uma boa solução on-premises.

Azure Container Registry

Embora ofereça serviços semelhantes ao AWS, o Azure Container Registry (ACR) da Microsoft possui suporte para imagens Docker e OCI, além de gráficos Helm.

O maior ponto de venda do Azure, no entanto, é sua geo-replicação de registro. Isso garante que todos tenham o mesmo acesso às imagens, não importa onde estejam, nas velocidades a que estão acostumados.

GitHub Container Registry

Dada a abrangência do GitHub e que este já está disponível para todos os usuários, o GitHub Container Registry (parte de um recurso maior chamado GitHub Packages) é uma das opções mais acessíveis.

Os benefícios de considerar o GitHub Packages para o gerenciamento de containers incluem:

- Gerenciamento de usuários simplificado - a maioria dos seus usuários já tem contas.
- Integração com GitHub Actions para enviar, publicar e implantar imagens.

- Seu custo em relação a outros serviços.

Google Container Registry

Enquanto outros serviços focam suas promessas em certos recursos importantes para um subconjunto de clientes prováveis, o Google Cloud Container Registry (GCR) é um sólido polivalente. Ele faz tudo o que você espera de um registro de contêineres e muito bem.

Assim como os outros grandes nomes em serviços em nuvem, você pode ficar preso(a) ao GCR, dependendo dos produtos do Google Cloud que usar. O Google Cloud Run, por exemplo, só usará imagens armazenadas no GCR, então lembre-se disso ao escolher um serviço de registro.

Ele não é tão cheio de recursos quanto o AWS ou o Azure, mas é uma oferta digna de um dos “Big 3” provedores de nuvem.

JFrog Container Registry

Construído sobre outro produto JFrog, o Artifactory, o JFrog Container Registry suporta imagens Docker e gráficos Helm. O JFrog Container Registry também oferece a opção de armazenar qualquer tipo de pacote, graças aos seus repositórios genéricos.

A JFrog tem opções tanto em nuvem quanto auto-hospedadas (ou híbridas, se preferir) e promete grande escalabilidade.

Red Hat Quay

Ao contrário de outras opções, o Red Hat Quay oferece apenas registros de containers privados. Isso o torna uma opção adequada especialmente para clientes em nível empresarial.

Agnóstico em relação ao provedor de nuvem, o Quay é fácil de conectar aos sistemas em qualquer extremidade do seu pipeline DevOps. Assim como o Azure, o Quay também inclui opções de geolocalização e, interessantemente, suporta BitTorrent para distribuição de containers.

A Red Hat também inclui uma solução de registro simplificada em sua plataforma Kubernetes, o OpenShift. No entanto, eles recomendam o Quay para equipes e organizações maiores.



O QUE VOCÊ VIU NESTA AULA?

Nessa aula você aprendeu o conceito de Images e Registries e sua importância, além de diferenciar containers de imagens. Você também aprendeu sobre a estrutura de imagens Docker e como criar uma imagem customizada e realizar upload da mesma em um dos registries mais famosos.

Além disso, você criou sua conta no Docker Hub e agora pode criar novas imagens Docker (ainda teremos uma aula para falar somente do Docker Hub). Agora você tem uma base sólida de conhecimento sobre Images e Registries, parabéns!

Lembre-se que estamos disponíveis na nossa comunidade do Discord para tirar dúvidas e te ajudar. Além disso, o conteúdo estará sempre disponível quando você sentir a necessidade de revisitá-lo!

REFERÊNCIAS

DOCKER DOCS. **Understanding the image layers**. 2024. Disponível em: <<https://docs.docker.com/guides/docker-concepts/building-images/understanding-image-layers/>>. Acesso em: 27 ago. 2024.

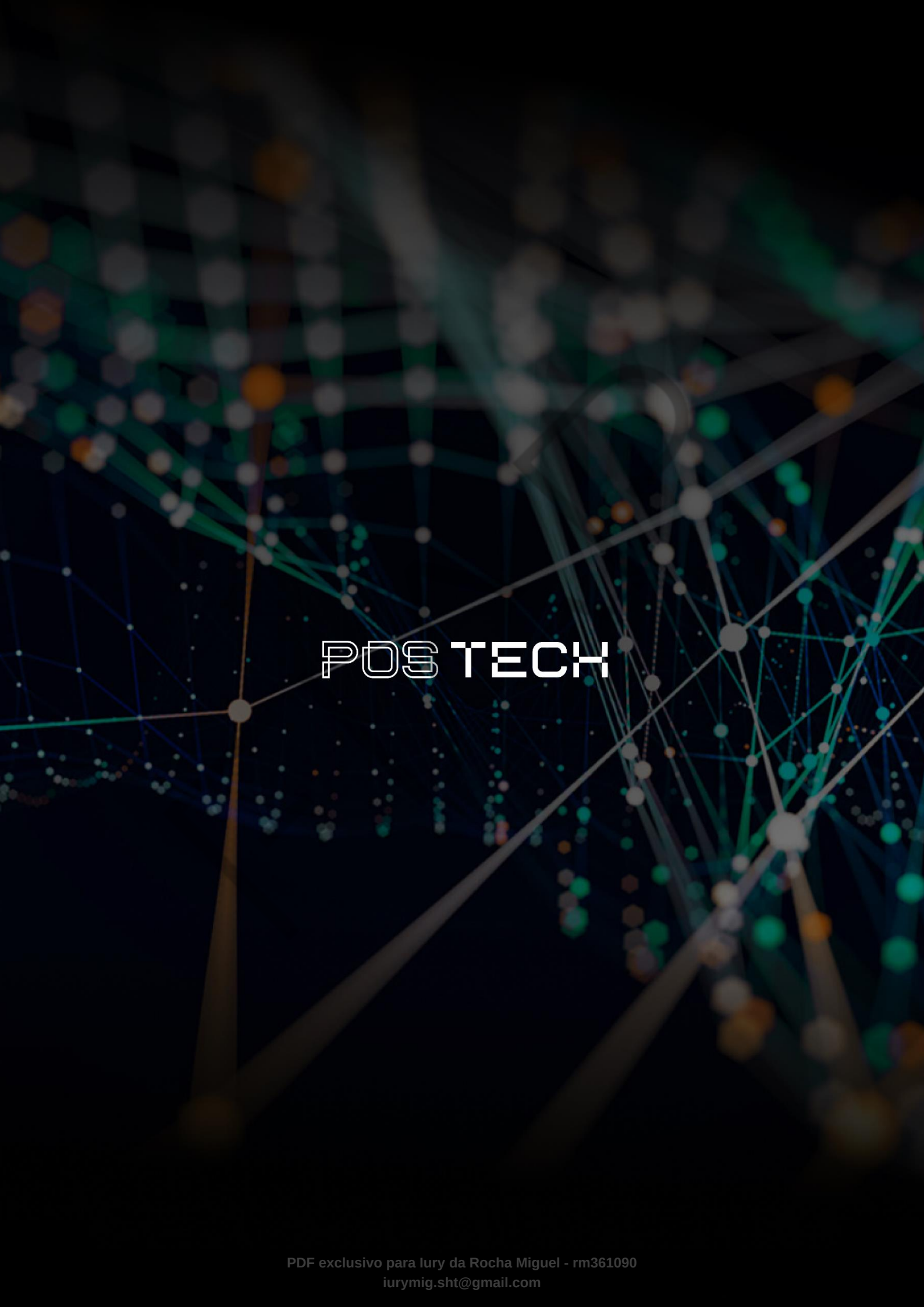
PAULO, W. **Docker e Docker Compose**. 2020. Disponível em: <<https://medium.com/@willianpaulo99/docker-e-docker-compose-bf967ffa84b3>>. Acesso em: 27 ago. 2024.

SUNDARAY, H. **Docker Architecture Explained**: Docker Client, Docker Daemon & Docker Registry. 2023. Disponível em: <<https://kodekloud.com/blog/docker-architecture/>>. Acesso em: 27 ago. 2024.

PALAVRAS-CHAVE

Palavras-chave: Registries. Docker. Containers. Criação de Imagens. Registry. Images Docker. Docker Hub.

EMSE



POSTECH